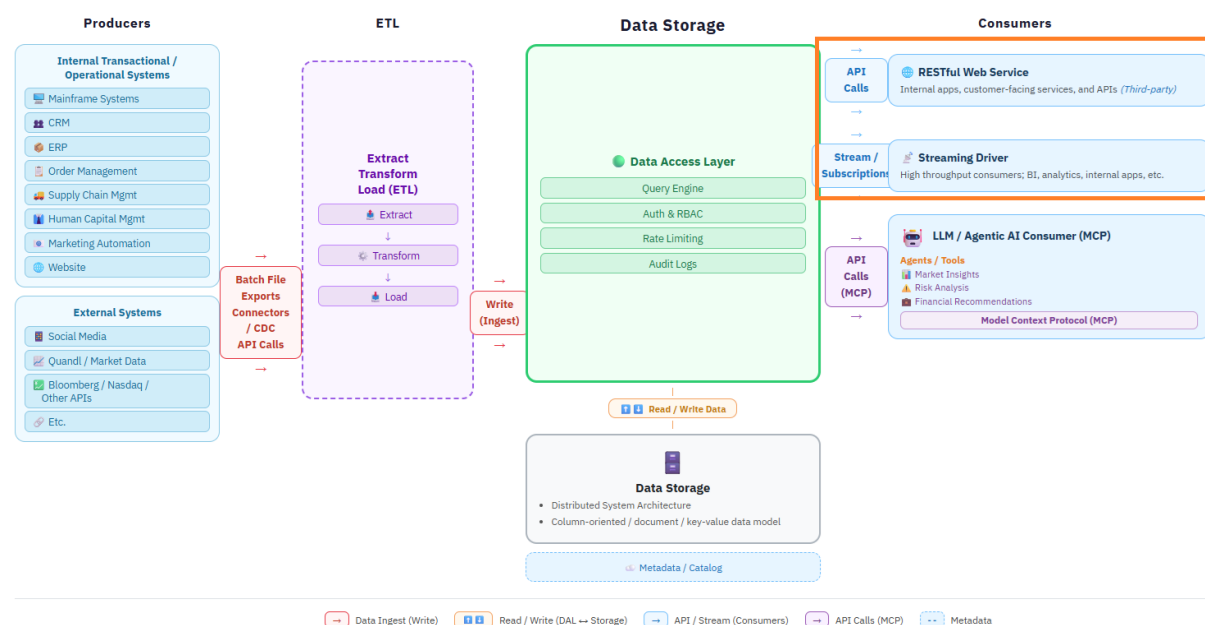


Consuming Data from Data Warehouse

Consuming Data from Data Warehouse	1
Designing Data Consumption Interfaces	1
Pagination Support in REST APIs	2
Endpoints Description	3
Streaming Client (optional)	5
Implementation Hints	6
Expected Deliverables	6
References	7

Designing Data Consumption Interfaces

In this lab, we move from storing data to making that data usable. A data warehouse is valuable only if other applications, analytics flows, or assistants can read from it in a clear and dependable way. The goal here is to design a consumption layer that lets clients (Web applications, mobile applications, desktop applications, MCP servers) discover assets, inspect their metadata, and retrieve time-series records without exposing internal storage details.



Learning outcomes

- Explain why data consumption must be designed differently from data storage.
- Design read-oriented interfaces for asset discovery, metadata lookup, and time-series retrieval.
- Use pagination to keep list and query responses predictable and scalable.

- Use streaming for long-running or high-volume reads so memory usage stays under control.

Because warehouse datasets grow quickly, the design must assume that some responses are too large to return in a single call. For that reason, this lab should be approached as an interface design exercise, not only as an API exposure task. Students should think carefully about how consumers move through data, how much data a client can safely handle at once, and how to preserve responsiveness when the volume increases.

Clients should be able to ask simple questions such as “what assets exist?”, “what do we know about this asset?”, “which data sources are available?”, and “give me the records for this asset in this interval”.

A good design separates client-facing concerns from persistence concerns. Consumers should not need to know how records are partitioned, how versions are stored, or how multiple physical reads are combined behind the scenes. The warehouse remains responsible for returning the right data, in the right order, with a response shape that is stable and easy to consume.

Because the project uses temporal data, the consumption layer must also define what “current” means. If multiple versions exist for the same logical record, the response should make it clear whether it returns the latest known version, full history, or data constrained to a business-date interval. These rules should be explicit and consistent across endpoints.

There are two possible approaches to expose large datasets for consumption, both discussed in the next sections:

- A paged interface implemented using REST API
- A streaming client

Pagination Support in REST APIs

Queries that may return many items must be pageable. In practice, that means the client never asks for “everything” when browsing assets, data sources, or long-time ranges. Instead, it asks for a limited window of results and continues step by step. This keeps response sizes reasonable, protects the server from oversized reads, and gives the client a predictable way to navigate large datasets.

REST is the acronym for Representational State Transfer, which is a set of constraints used to define Web service-based software architectures. Web services using the REST architectural style [REST], i.e. *RESTful* Web services, provide interoperability between computer systems on the Internet. RESTful Web services allow the requesting systems to access and manipulate textual representations of Web resources by using a uniform and predefined set of stateless operations.

In a RESTful Web service, requests made to a resource's URI will elicit a response with a payload formatted in JSON/XML/HTML or some other format. When HTTP is used, as it is most common, the operations (HTTP methods) available are GET, POST, PUT, PATCH, DELETE, HEAD, CONNECT, OPTIONS and TRACE.

HTTP-based RESTful APIs are defined with the following aspects:

- a base URI, such as <http://api.example.com/collection/>, also called endpoint
- standard HTTP methods (e.g., GET, POST, PUT, PATCH and DELETE)
- a media type that defines state transition data elements (e.g., JSON). The current representation tells the client how to compose requests for transitions to all the next available application states.

Because our endpoints may return large amount of data, we need to *paginate* the data so that clients can handle it gracefully. There are multiple approaches to pagination, such as:

- 1) **User-defined pagination:** the Limit/Offset approach became popular with server-side applications using SQL databases which natively support LIMIT and OFFSET as part of the SQL SELECT syntax. An endpoint which uses user-defined pagination looks like

```
GET /items?limit=20&offset=100
```

which returns 20 items starting with the position 100. Note that the size of the page is controlled by the client.

- 2) **Server-defined pagination:** such an endpoint looks like

```
GET /items?page=1
```

which returns first page of items. The size of the page and the windowing of the dataset is controlled by the server. Clients will simply iterate through pages (page=2, page=3, etc.) until they get an empty page.

- 3) **Cursor-based pagination:**

```
GET /items?nextCursorId=<next page id>
```

This is the approach used by Nasdaq Data Link [NDL]; the server returns a `nextCursorId` alongside with each page of data (or null if no more pages are available), which will be echoed by clients as a query parameter to get the next page of data. Comparing to the previous approaches, clients are not able to fetch random pages (e.g., page 3, page 10, or from item 100 to item 120) of data and they need to navigate/fetch the data page by page. The page size can be either server-defined or controlled by clients via an additional query parameter (number of items).

The endpoints below were defined using a *user-defined pagination* approach, where users will control the amount of data received in one call. If you like/want another approach better, feel free to implement it.

Note: `BASE_URL` – indicates the base URL of the REST API. This may differ from one implementation to another and may contain route components describing the version.

Example: <http://localhost:8080/api/v1> - this is the version 1 of the API exposed on port 8080 of localhost domain.

Endpoints Description

- 1) **/assets** – returns the assets available in the data warehouse

1.1) GET <BASE_URL>/assets – returns a JSON array of **ids** (id = partition key of the table) of all assets available in the data warehouse

Parameters:

- Offset – starting position in the collection of all ids (optional; default = 0)
- Limit – maximum number of returned asset ids (optional; default = 20)

Example:

GET <BASE_URL>/assets – returns the first 20 asset ids in alphabetical order as a JSON array ([“QDL/BITFINEX/BTCUSD”,... “QDL/BITFINEX/ZRXUSD”])

GET <BASE_URL>/assets&offset=30 – returns the next 20 asset ids starting at position 30 in alphabetical order as a JSON array ([“QDL/BITFINEX/BTCUSD”, “QDL/BITFINEX/SOLUSD”])

1.2) GET <BASE_URL>/assets/{assetId} – returns the detailed JSON representation for the asset with specified assetId

Implementation note: For an assetId there may be multiple entries in asset table, with different *system_times*; getting the most recent one can be implemented either as an in-memory filtering or leveraging table’s primary key / clustering columns ordering.

Example: GET <BASE_URL>/assets/QDL/BITFINEX/BTCUSD – returns

```
{ { "id": "QDL/BITFINEX/BTCUSD", "system_time": "2025-04-04 01:00:00", "attributes": { "attr_1": "updated value" } }, { "id": "QDL/BITFINEX/BTCUSD", "system_time": "2023-04-05 02:01:00", "attributes": { "attr_1": "some value", "attr_2": "some value" } }
```

2) **/data-sources** – returns the list of data sources’ ids and the detail of a specified data source, respectively

GET <BASE_URL>/data-sources

GET <BASE_URL>/data-sources/{dataSourceId}

The behaviour should be similar to /assets endpoint.

3) **/data** – returns the time series data for a specified asset and data source

GET
<BASE_URL>/data?assetId={assetId}&dataSourceId={dataSourceId}&startBusinessDate=2018-01-01&endBusinessDate=2018-02-01&includeAttributes=false

Query parameters:

- assetId = asset id
- dataSourceId = data source id
- startBusinessDate = the oldest date of time series records (format YYYY-MM-DD)
- endBusinessDate = the newest date of time series records (format YYYY-MM-DD)
- includeAttributes = indicates whether the response will include the list of attributes (boolean value true/false)

This endpoint returns all timeseries records between [startBusinessDate, endBusinessDate) (start – included, end – not included) for the asset with id = assetId and data source dataSourceId.

Note 1: if for a business date in the range there are multiple records (with different system_time) then only the most recent one should be included in the response!

Note 2: the records are returned in descending chronological order of business_date, i.e. newest first.

Note 3: if includeAttributes = true then an array with attributes description is returned as well

```
{
  "data": {
    "assetId": assetId,
    "dataSourceId": dataSourceId,
    "records": [{
      "businessDate": "2018-01-05",
      "values": [{"attr_id1": value11, "attr_id2": value21}]
    }, {
      "businessDate": "2018-01-06",
      "values": [{"attr_id1": value12, "attr_id3": value32}]
    }]
  }
  "attributes": ["attr_id1", "attr_id2", "attr_id3"]
}
```

Streaming Client (optional)

The streaming client (included as Streaming Driver in the architecture diagram) is optional. A useful rule of thumb to determine whether to use pagination or streaming is this: use pagination when the client explores or browses a collection and use streaming when the client needs to process a potentially long result flow. The two principles complement each other, and a mature platform will often support both. When processing or delivering large amounts of data, the platform should favour streaming over loading everything into memory first.

Streaming means results can be produced and consumed progressively. That is especially useful for long time-series ranges, exports, analytics pipelines, and internal processing steps that pass records from one stage to another. If the platform claims to support large reads, then the path from API contract down to database fetching should be designed for continuous flow. The service should read manageable chunks, transform records incrementally, and send or hand off results as they become available. This keeps memory usage low and allows downstream consumers to start working before the full dataset has finished loading.

Design hint: streaming should exist end to end:

- At API level, define endpoints or response modes that can return data progressively when result sets are large.
- At service level, process records in batches or iterators rather than collecting the full set first.
- At persistence level, fetch rows incrementally so the data access path does not accumulate the whole result in memory.
- For analytics or exports, pass records forward continuously instead of building one giant intermediate collection.
- Document when a client should use paging and when a client should use streaming; both solve scale, but for different usage patterns.

Implementation Hints

- Pagination must exist end to end.
 - At API level, expose request parameters or tokens that let clients request only one slice of data at a time.
 - At service level, keep page metadata together with the returned results so clients know how to continue.
 - At persistence level, issue bounded reads that naturally support page-by-page traversal.
 - Prefer deterministic ordering for pageable collections, so repeated reads do not shuffle results unexpectedly.
 - Avoid in-memory pagination over a full dataset except for tiny demonstrations; it defeats the point of the design!!
- The API is about consumption, so avoid turning every endpoint into a generic CRUD surface.
- Return compact summaries in list endpoints and detailed payloads only in detail endpoints. This reduces unnecessary transfer and parsing.
- For heterogeneous time-series values, keep the response model flexible enough to support different attribute sets without breaking clients.
- Java / Apache Cassandra implementation suggestions
 - Use Spring Data Cassandra Repository's support for paginated and sorted access to the entities; more about `Slice<T>` in [Slice]
 - Build native, custom CQL queries to paginate access to Access tables; see [Paging] for more details

Expected Deliverables

- A documented (e.g., using OpenAPI) set of endpoints or equivalent service contracts for assets, data sources, and time-series reads.
- Explicit support for pagination in queries that may return many results.
- (optional) Explicit support for streaming in large-data processing or delivery flows.
- A brief explanation of how the platform returns the correct latest version when multiple temporal records exist.

References

[REST] <https://restfulapi.net/>

[Slice] <https://docs.spring.io/spring-data/cassandra/reference/cassandra/repositories/repositories.html>

[Paging] <https://community.ibm.com/community/user/dataexchange/blogs/tanvi-kakodkar1/2020/01/24/paging-filtering-apache-cassandra>

[NDL] <https://docs.data.nasdaq.com/docs/in-depth-usage-1#pagination>